

Tutorial: Implementando Autenticação e JWT no Backend Node.js

Este tutorial ensina o passo a passo de como proteger rotas da sua API usando JSON Web Tokens (JWT) e criptografia de senhas. **Atenção:** Este material aborda apenas a configuração do Backend.

1. Instalando Dependências

No terminal, dentro da pasta do seu projeto, instale as bibliotecas necessárias:

```
npm install jsonwebtoken bcryptjs
```

- **jsonwebtoken:** Cria e valida o "crachá" (Token) do usuário.
 - **bcryptjs:** Embaralha (criptografa) a senha no banco de dados para que ninguém consiga ler a senha real.
-

2. Preparando o Banco de Dados

Adicione a tabela de usuários ao seu script SQL (ex: `database.sql`) e execute-o no seu SGBD (MySQL Workbench, DBeaver, etc):

```
CREATE TABLE IF NOT EXISTS usuario (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  nome VARCHAR(100) NOT NULL,  
  email VARCHAR(100) NOT NULL UNIQUE,  
  senha VARCHAR(255) NOT NULL,  
  papel ENUM('admin', 'cliente') DEFAULT 'cliente',  
  criado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

3. Criando a Camada MVC do Usuário

A. Repository (`src/repositories/UsuarioRepository.js`)

Crie as funções que conversam direto com a tabela `usuario` usando seu arquivo de conexão (`pool`):

1. `create({nome, email, senha, papel})`: Faz o `INSERT INTO`.
2. `findByEmail(email)`: Faz um `SELECT` buscando pelo e-mail, útil para validar login e evitar contas duplicadas.

B. Service (`src/services/UsuarioService.js`)

Aqui você deve implementar duas funções principais:

1. **registrarUsuario()**:

- Valida se o email já existe usando o Repository.
- Gera o Hash da senha com **bcrypt**:

```
const salt = await bcrypt.genSalt(10);
const senhaHash = await bcrypt.hash(senha, salt);
```

- Salva o usuário chamando o Repository.

2. **login()**:

- Busca o usuário pelo e-mail.
- Compara a senha digitada com a criptografada:

```
const senhaCorreta = await bcrypt.compare(senhaDigitada,
usuario.senha);
```

- Se for correta, assina um JWT devolvendo o token:

```
const token = jwt.sign({ id: usuario.id, papel: usuario.papel },
'sua_chave_secreta', { expiresIn: '8h' });
```

C. Controller (**src/controllers/UsuarioController.js**)

Crie os métodos estáticos **registrar(req, res)** e **login(req, res)** para receber os dados do **req.body**, repassar ao Service e devolver um **res.status(200).json()** com o token.

4. Criando as Rotas Abertas (Login/Registro)

Crie um arquivo **src/routes/authRoutes.js**:

```
const express = require('express');
const router = express.Router();
const UsuarioController = require('../controllers/UsuarioController');

router.post('/registrar', UsuarioController.registrar);
router.post('/login', UsuarioController.login);

module.exports = router;
```

(Não se esqueça de importar esse arquivo no `app.js` ou no `index.js` de rotas com `app.use('/auth', authRoutes)`).

5. Criando o Middleware de Proteção ("O Segurança")

Crie o arquivo `src/middlewares/authMiddleware.js`. O papel dele é interceptar todas as requisições que precisem de segurança e checar o Token antes de deixar chegar ao Controller:

```
const jwt = require('jsonwebtoken');

const verificarToken = (req, res, next) => {
  // 1. Pega o cabeçalho "Authorization: Bearer <token>"
  const authHeader = req.headers.authorization;
  if (!authHeader) return res.status(401).json({ mensagem: "Token não fornecido" });

  const token = authHeader.split(' ')[1]; // Extrai só a parte do código

  try {
    // 2. Tenta decodificar o token com a mesma senha secreta
    const decodificado = jwt.verify(token, 'sua_chave_secreta');
    req.usuarioPapel = decodificado.papel;
    return next(); // Se estiver tudo ok, deixa a requisição passar!
  } catch (err) {
    return res.status(401).json({ mensagem: "Token inválido" });
  }
};

const verificarAdmin = (req, res, next) => {
  // Esse middleware deve ser colocado DEPOIS do verificarToken
  if (req.usuarioPapel !== 'admin') {
    return res.status(403).json({ mensagem: "Acesso restrito para administradores" });
  }
  return next();
};

module.exports = { verificarToken, verificarAdmin };
```

6. Trancando a Porta! (Protegendo as Rotas de Produto)

Vá no arquivo `src/routes/produtoRoutes.js`. Importe os middlewares e adicione eles no meio da declaração da rota de exclusão/criação:

```
const { verificarToken, verificarAdmin } =
  require('../middlewares/authMiddleware');

// A listagem de produtos continua livre:
```

```
router.get('/', ProdutoController.listar);

// Mas a criação e exclusão agora exigem o Token + Permissão de Admin:
router.post('/', verificarToken, verificarAdmin, ProdutoController.cadastrar);
router.delete('/:id', verificarToken, verificarAdmin, ProdutoController.deletar);
```

Pronto! Sua API agora tem um sistema completo de permissões e controle de acesso!